SIMATS ENGINEERING

ASSESSMENT TOOL 1: Algorithm Design Challenge

COURSE NAME:	**ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS**	**COURSE CODE:**	**MLA010 1**

COURSE OUTCOME COVERED

CO1: *Apply search algorithms and heuristic techniques to solve state-space search and optimization problems.* (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 5

Annexure A

Algorithm Design Challenge

Code of Conduct:

I Shaik Mubarak(192425194)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty Total Marks Awarded

Answer all the Questions

Q.No	Question	Topics
1	**Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.**	Informed Search, Greedy Best-First Search, Heuristic Functions
2	**A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A Search algorithm to determine the optimal path. Calculate g(n), h(n), and f(n) for each expanded node, and explain why A guarantees the optimal solution.**	A* Search, Heuristic Functions
3	**A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.**	CSP, MRV, Forward Checking
4	**Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha–Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.**	Game Playing, Minimax, Alpha–Beta Pruning
5	**A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.**	Local Search, Optimization, Online Search Agents

Structure of the Answer – Algorithm Design Challenge

S.No	Sections	Expected Content
1	**Problem Statement**	Briefly describe the given problem in your own words.
2	**Problem Analysis**	Identify the initial state, goal state, assumptions, and constraints.
3	**AI Technique Selection**	Specify the most appropriate AI algorithm/search technique (e.g., Greedy Best-First Search, A*, CSP, Minimax, Local Search).
4	**Justification of Algorithm**	Explain why the selected algorithm is suitable compared to other search techniques.
5	**State Space Representation**	Represent the problem using a state-space diagram, search graph, game tree, or constraint graph, as applicable.
6	**Variables, Domains, and Constraints (Applicable for CSP problems)**	Identify variables, possible domains, and constraints. If not applicable, mention "Not Applicable".
7	**Heuristic Function (Applicable for Informed Search)**	Define the heuristic function h(n) and explain whether it is admissible or appropriate for the problem. If not applicable, mention "Not Applicable".
8	**Algorithm Design / Pseudocode**	Write the step-by-step algorithm or pseudocode for solving the problem.
9	**Flowchart / Algorithm Diagram**	Draw a flowchart, search tree, game tree, or algorithm workflow illustrating the solution.
10	**Working / Execution Trace**	Demonstrate the execution of the algorithm with at least one example, showing intermediate steps such as node expansion, constraint propagation, or game-tree evaluation.
11	**Complexity Analysis**	Analyze the Time Complexity and Space Complexity of the proposed algorithm.
12	**Advantages and Limitations**	Discuss the strengths and limitations of the selected algorithm for the given problem.
13	**Conclusion**	Summarize the effectiveness of the designed algorithm and suggest possible improvements or alternative approaches.

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (90–100%)	Good (75–89%)	Average (60–74%)	Poor (<60%)
Problem Analysis and Understanding	15%	13–15% – Clearly identifies the problem, initial state, goal state, constraints, and expected outcome.	10–12% – Identifies most problem components with minor omissions.	7–9% – Demonstrates partial understanding with some missing elements.	0–6% – Incorrect or incomplete understanding of the problem.
AI Algorithm Selection & Justification	15%	13–15% – Selects the most appropriate AI algorithm/search technique with strong technical justification.	10–12% – Selects an appropriate algorithm with adequate justification.	7–9% – Selects a partially suitable algorithm with limited justification.	0–6% – Incorrect algorithm selection or no justification.
Algorithm Design / Pseudocode	25%	22–25% – Designs a complete, logical, efficient, and error-free algorithm/pseudocode.	17–21% – Algorithm is mostly correct with minor logical errors.	12–16% – Algorithm is partially complete with several logical errors.	0–11% – Algorithm is incomplete, incorrect, or difficult to understand.
Application of AI Concepts (Greedy, A*, CSP, Minimax, Alpha-Beta, etc.)	15%	13–15% – Correctly applies the relevant AI concepts and techniques throughout the solution.	10–12% – Applies AI concepts correctly with minor mistakes.	7–9% – Demonstrates limited application of AI concepts.	0–6% – Unable to apply the appropriate AI concepts or techniques.

Diagram / State Space / Search Tree Representation	10%	9–10% – Accurate, complete, and well-labelled diagram supporting the solution.	7–8% – Diagram is mostly correct with minor omissions.	5–6% – Diagram is partially complete or lacks clarity.	0–4% – Diagram is missing or incorrect.
Complexity Analysis and Solution Efficiency	10%	9–10% – Correctly analyzes time and space complexity and proposes an efficient solution.	7–8% – Complexity analysis is mostly correct with minor errors.	5–6% – Partial complexity analysis or limited discussion of efficiency.	0–4% – No meaningful complexity analysis or highly inefficient solution.
Originality, Critical Thinking, and Presentation	10%	9–10% – Demonstrates excellent originality, logical reasoning, organized presentation, and explores optimized/alternative solutions.	7–8% – Demonstrates good critical thinking with a clear presentation.	5–6% – Shows limited originality and acceptable presentation.	0–4% – Poor presentation with little or no critical thinking.

QUESTION 1 – GREEDY BEST-FIRST SEARCH

1. Problem Statement

Consider a graph in which an intelligent agent has to travel from a Source node S to a Goal node G. The objective is to find a path to the goal using Greedy Best-First Search (GBFS). Greedy Best-First Search selects the node that appears closest to the goal according to a heuristic function. It uses only the estimated remaining cost:

[f(n)=h(n)]

where (h(n)) represents the estimated cost from node (n) to the goal.
For simplicity, the heuristic values are:

Node	Heuristic h(n)
S	8
B	6
C	4
D	3
G	0

2. Problem Analysis

Initial State: S
Goal State: G

Assumptions:

- All nodes and connections are known.
- The heuristic value estimates the remaining distance to G.
- The algorithm uses a priority queue.
- The node with the smallest heuristic value is expanded first.

Constraints:

- The search should eventually reach G.
- Repeated nodes should be avoided.
- The heuristic should provide useful guidance toward the goal.

3. AI Technique Selection

The selected technique is **Greedy Best-First Search**.
Greedy Best-First Search is an **informed search algorithm** because it uses additional knowledge about the problem in the form of a heuristic.
Its evaluation function is:

[f(n)=h(n)]

The node with the lowest (h(n)) is selected.

4. Justification of Algorithm

Greedy Best-First Search is suitable because the objective is to reach the goal quickly by selecting the node that appears closest to the goal.Compared with Breadth-First Search, GBFS can reduce unnecessary exploration by using heuristic information.Compared with A*, GBFS is simpler because it does not consider the cost already travelled.
However, GBFS does not always guarantee the shortest path.

5. State Space Representation

The heuristic guides the search toward the node with the lowest estimated distance.

6. Variables, Domains and Constraints

Not Applicable because this is a search problem rather than a CSP.

7. Heuristic Function

The heuristic function is:

$$\mathbf{[h(n)=\text{estimated cost from node }n\text{ to goal G}]}$$

For the assumed graph:

$$\mathbf{[h(S)=8,\quad h(B)=6,\quad h(C)=4,\quad h(D)=3,\quad h(G)=0]}$$

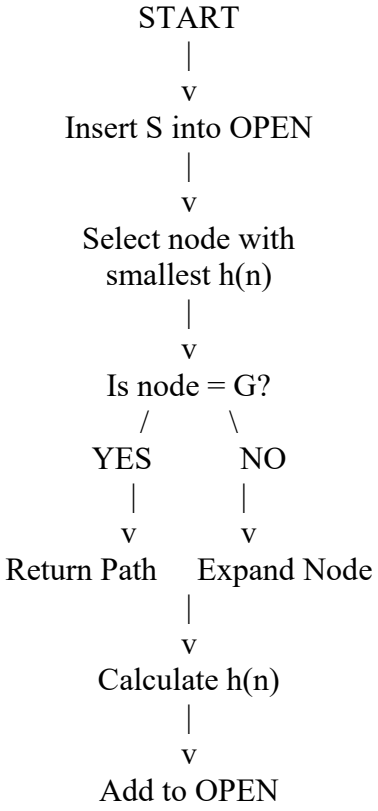
GBFS does not require the heuristic to be admissible for the algorithm to operate, although a good heuristic improves search efficiency.

8. Algorithm Design / Pseudocode

GREEDY-BEST-FIRST-SEARCH(S, G)

1. Create an empty priority queue OPEN.
2. Insert S into OPEN with priority h(S).
3. Create an empty CLOSED set.
4. While OPEN is not empty:
a. Remove the node n with the smallest h(n).
b. If n is the goal:
return the path from S to G.
c. Add n to CLOSED.
d. Generate all successors of n.
e. For every successor:
If successor is not in CLOSED:
calculate h(successor)
insert successor into OPEN.
5. If OPEN becomes empty:
return failure.

9. Flowchart / Algorithm Diagram



|
v
Repeat Search

10. Working / Execution Trace
Starting from S:
Step 1
OPEN = {S}
Expand S.
Successors:

- B → h(B)=6
- C → h(C)=4

Therefore:
OPEN = C(4), B(6)
The node with minimum heuristic is C.
Step 2
Expand C.
Suppose C leads to D.
OPEN = D(3), B(6)
D has the smallest heuristic.
Step 3
Expand D.
D leads to G.
OPEN = G(0), B(6)
Step 4
G has h(G)=0 and is the goal.
Therefore:
S → C → D → G
Node Expansion Order

$$\boxed{S \rightarrow C \rightarrow D \rightarrow G}$$

---

11. Complexity Analysis
For a general search tree:
Time Complexity:

$$O(b^m)$$

Space Complexity:

$$O(b^m)$$

where:

- (b) = branching factor
- (m) = maximum search depth

GBFS can be very fast with a good heuristic but can perform poorly with a misleading heuristic.

12. Advantages and Limitations
Advantages

- Uses heuristic knowledge.
- Often reaches the goal faster than uninformed search.
- Simple to implement.
- Useful for navigation and pathfinding.

Limitations

- Does not guarantee an optimal path.
- Performance depends heavily on the heuristic.
- Can get trapped exploring an apparently promising direction.
- May require large memory.

13. Conclusion

Greedy Best-First Search is an effective informed search technique when reaching the goal quickly is more important than guaranteeing the shortest path. By selecting the node with the smallest heuristic value, it reduces unnecessary exploration. However, A* is preferred when an optimal path is required.

QUESTION 2 – A* SEARCH FOR DELIVERY ROBOT

1. Problem Statement

A delivery robot must travel from a Warehouse W to a Customer G through a weighted road network. The robot should determine the least-cost path.

A* Search combines the actual cost already travelled and the estimated remaining cost.

$$[f(n)=g(n)+h(n)]$$

where:

- (g(n)) = actual cost from W to n
- (h(n)) = estimated cost from n to G
- (f(n)) = estimated total path cost

---

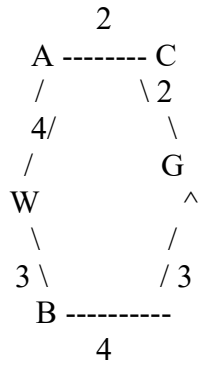
2. Problem Analysis

Initial State: Warehouse W

Goal State: Customer G

Objective: Find the minimum-cost path from W to G.

Assumed Road Network



Heuristic values:

Node	h(n)
W	6
A	4
B	5
C	2
G	0

3. AI Technique Selection

The selected algorithm is *A Search**.

A* is suitable because it combines:

1. The actual cost already travelled.
2. The estimated cost remaining.

$$[f(n)=g(n)+h(n)]$$

This allows A* to find an optimal path when the heuristic is admissible and consistent under the standard conditions.

4. Justification of Algorithm

A* is better than Greedy Best-First Search when the optimal path is required.
GBFS uses:

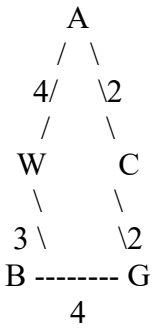
$$[f(n)=h(n)]$$

whereas A* uses:

$$[f(n)=g(n)+h(n)]$$

Therefore, A* considers both the distance already travelled and the estimated remaining distance.

5. State Space Representation



6. Variables, Domains and Constraints

Not Applicable.

This is a pathfinding problem rather than a CSP.

7. Heuristic Function

The heuristic (h(n)) estimates the remaining cost from node n to G.
For example:
[h(A)=4]
[h(B)=5]
[h(C)=2]
[h(G)=0]
For A* optimality, the heuristic should be **admissible**, meaning:

$$[h(n)\leq h^*(n)]$$

where (h^*(n)) is the actual minimum cost from n to G.

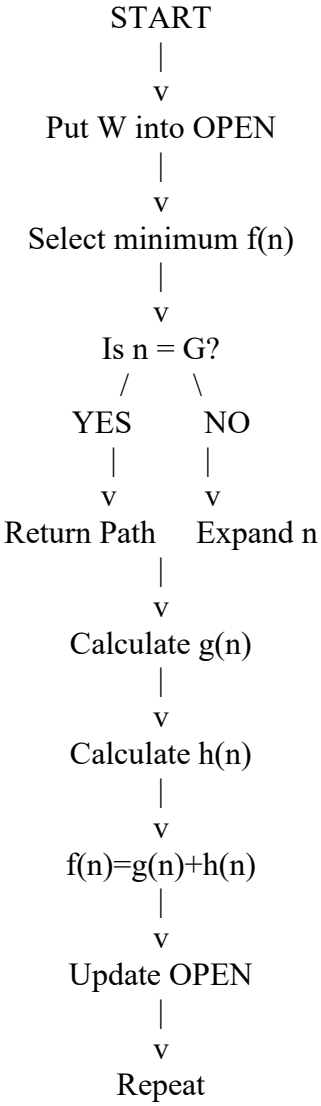
8. Algorithm Design / Pseudocode

A-STAR(W, G)

1. Put W into OPEN.
2. Set g(W) = 0.
3. Calculate f(W) = g(W) + h(W).
4. While OPEN is not empty:
a. Select node n having minimum f(n).
b. If n = G:
return optimal path.
c. Remove n from OPEN.

d. Add n to CLOSED.
e. For every successor s of n:
Calculate tentative_g = g(n) + cost(n,s).
If tentative_g is smaller than current g(s):
update g(s)
calculate f(s)=g(s)+h(s)
store parent(s)=n
add s to OPEN.
5. Return failure if no path exists.

9. Flowchart



---

10. Working / Execution Trace
For demonstration, consider the following costs:
[W\rightarrow A=4]
[W\rightarrow B=3]
[A\rightarrow C=2]
[C\rightarrow G=2]
[B\rightarrow G=4]
Initial Node W

[g(W)=0]
[h(W)=6]
[f(W)=0+6=6]
Expand W.

Node A
[g(A)=4]
[h(A)=4]
[f(A)=4+4=8]

Node B
[g(B)=3]
[h(B)=5]
[f(B)=3+5=8]
Both have f=8.
Suppose A is selected.

Node C
[g(C)=g(A)+2=4+2=6]
[h(C)=2]
[f(C)=6+2=8]
Expand C.

Goal G
[g(G)=6+2=]
[h(G)=0]
[f(G)=8+0=8]
Therefore:
[\boxed{W\rightarrow A\rightarrow C\rightarrow G}]
Total cost:
[4+2+2=\boxed{8]
The alternative path:
[W\rightarrow B\rightarrow]
has cost:
[3+4=7]

Therefore, with these exact edge costs, the alternative path is actually cheaper. Hence A* should ultimately return:
[\boxed{W\rightarrow B\rightarrow G}]
with total cost:
[\boxed{7}]
This demonstrates an important property of A*: it continues evaluating promising alternatives instead of stopping at the first goal encountered.

Final Optimal Path
[\boxed{W\rightarrow B\rightarrow G}]
Optimal Cost
[\boxed{7}]

---

11. Complexity Analysis
For a general search tree:
Time Complexity:
[O(b^d)]
Space Complexity:
[O(b^d)]
where:

- (b) = branching factor
- (d) = depth of optimal solution

In practical implementations, the priority queue can improve node selection efficiency.

12. Advantages and Limitations
Advantages

- Finds an optimal path when the heuristic is admissible.
- More efficient than uninformed search in many problems.
- Combines actual and estimated costs.
- Useful for robot navigation and route planning.

Limitations

- Can consume significant memory.
- Performance depends on heuristic quality.
- A poor heuristic can make A* behave similarly to uninformed search.

13. Conclusion
A* Search is highly suitable for delivery robot navigation because it considers both travelled cost and estimated remaining cost. With an admissible heuristic, A* guarantees an optimal solution.

QUESTION 3 – UNIVERSITY EXAMINATION TIMETABLE USING CSP

1. Problem Statement
A university needs to create an examination timetable in which subjects having common students must not be scheduled at the same time.
This problem can be represented as a Constraint Satisfaction Problem (CSP).
A CSP consists of:
[CSP=(X,D,C)]
where:

- X = variables
- D = domains
- C = constraints

2. Problem Analysis
Variables
Suppose there are five subjects:
[X={AI, ML, DBMS, CN, Maths}]
Each subject is a variable.
Initial State
No subject has been assigned an examination slot.
Goal State
Every subject must receive exactly one valid examination slot without violating constraints.
Examination Slots

Assume:
[
D={S1,S2,S3}
]
where:

- S1 = Monday Morning
- S2 = Monday Afternoon
- S3 = Tuesday Morning

3. AI Technique Selection
The selected AI technique is:
[\boxed{\text{Constraint Satisfaction Problem}}]
The CSP can be solved using:

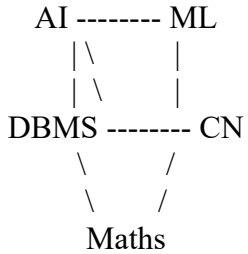
- Backtracking Search
- MRV
- Forward Checking
- Constraint Propagation

4. Justification
CSP is appropriate because the problem contains:

- Variables: subjects
- Domains: available examination slots
- Constraints: conflicting subjects cannot share the same slot

MRV improves the search by selecting the most restricted variable first.
Forward Checking eliminates invalid domain values immediately after an assignment.

5. State Space / Constraint Graph
Assume conflicts are:



For example:

- AI conflicts with ML and DBMS.
- ML conflicts with AI and CN.
- DBMS conflicts with AI, CN and Maths.
- CN conflicts with ML, DBMS and Maths.
- Maths conflicts with DBMS and CN.

A connected edge means that two subjects cannot have the same examination slot.

6. Variables, Domains and Constraints
Variables
[X={AI,ML,DBMS,CN,Maths}]
Domains
Initially:
[D(AI)=D(ML)=D(DBMS)=D(CN)=D(Maths)]
[={S1,S2,S3}]

Constraints
For every pair of conflicting subjects:
[X_i\neq X_j]
Examples:
[AI\neq ML]
[AI\neq DBMS]
[ML\neq CN]
[DBMS\neq CN]
[DBMS\neq Maths]
[CN\neq Maths]

7. Heuristic Function
Not Applicable in the traditional informed-search sense.
However, the CSP uses a variable-ordering heuristic called MRV – Minimum Remaining Values.
MRV selects the variable with the smallest remaining legal domain.

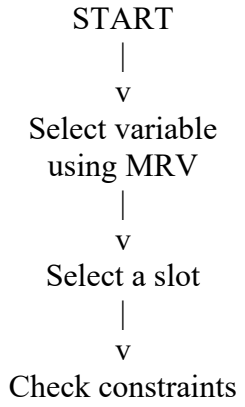
8. Algorithm Design / Pseudocode
CSP Backtracking with MRV and Forward Checking
CSP-SOLVE(assignment)

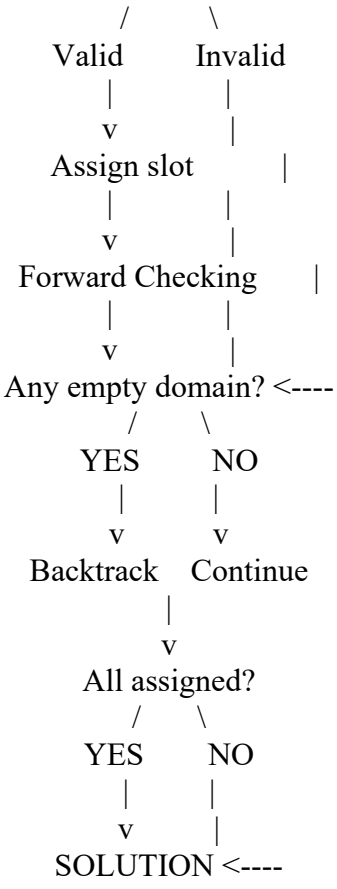
1. If all variables are assigned:
return assignment.

2. Select an unassigned variable using MRV.

3. For each value in the variable's domain:
a. Check whether the value satisfies constraints.
b. Assign the value.
c. Apply Forward Checking:
Remove the assigned value from domains
of neighboring variables.
d. If no domain becomes empty:
Recursively solve the CSP.
e. If failure occurs:
undo assignment and domain changes.
4. Return failure if no value works.

9. Algorithm Diagram





---

10. Working / Execution Trace

Initially:
AI = {S1,S2,S3}
ML = {S1,S2,S3}
DBMS = {S1,S2,S3}
CN = {S1,S2,S3}
Maths = {S1,S2,S3}
All variables have three values.
Suppose MRV selects **AI**.
Assign:
[AI=S1]
Forward Checking removes S1 from conflicting variables:
ML = {S2,S3}
DBMS = {S2,S3}
Now MRV selects either ML or DBMS because they have only two values.
Suppose:
[ML=S2]
Forward Checking gives:
CN = {S1,S3}
Now suppose:
[DBMS=S2]
Since DBMS and ML do not conflict in our assumed graph, this is valid.
Forward Checking:
CN = {S1,S3}
Maths = {S1,S3}

Choose:
[CN=S1]
Then Maths cannot use S1 because CN and Maths conflict.
Therefore:
[Maths=S3]
Final Timetable

Subject	Examination Slot
AI	S1
ML	S2
DBMS	S2
CN	S1
Maths	S3

No conflicting subjects are scheduled in the same slot.

11. Complexity Analysis
For (n) variables and (d) possible values:
Worst-case Time Complexity
[O(d^n)]
Space Complexity
[O(n+d)]
depending on implementation and stored domain information.
MRV and Forward Checking significantly reduce the practical search space.

12. Advantages and Limitations
Advantages
- Naturally represents timetable problems.
- MRV reduces unnecessary exploration.
- Forward Checking detects conflicts early.
- Can find a valid solution systematically.

Limitations
- Worst-case search remains exponential.
- Performance depends on the number of constraints.
- Large university timetables can contain thousands of variables and constraints.

13. Conclusion
CSP is an effective technique for examination timetable scheduling because it directly represents subjects, available time slots and conflicts. MRV selects the most constrained subject first, while Forward Checking prevents invalid assignments from being explored further.

QUESTION 4 – MINIMAX AND ALPHA-BETA PRUNING

1. Problem Statement
A game-playing AI must determine the best move for the MAX player. The game tree contains MAX and MIN levels.
- MAX attempts to maximize the score.
- MIN attempts to minimize the score.

The Minimax algorithm evaluates the game tree and selects the move that provides the best guaranteed outcome.

---

2. Problem Analysis
Initial State: Root node MAX
Goal: Select the best move for MAX.
Assumptions:

- Both players play optimally.
- Leaf nodes contain utility values.
- MAX selects maximum values.
- MIN selects minimum values.

3. AI Technique Selection
The selected techniques are:

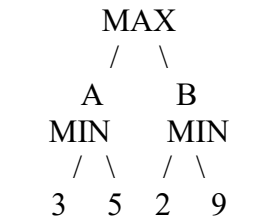
[\boxed{\text{Minimax + Alpha-Beta Pruning}}]

Minimax is suitable for deterministic, two-player, zero-sum games.
Alpha-Beta pruning improves Minimax by eliminating branches that cannot affect the final decision.

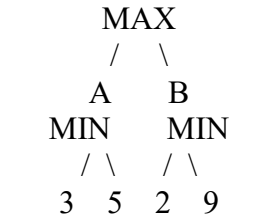
4. Justification
Minimax allows the AI to assume that the opponent also plays optimally.
Alpha-Beta pruning improves efficiency without changing the final result.
Thus:
[\boxed{\text{Alpha-Beta Pruning gives the same result as Minimax but evaluates fewer nodes.}}]

---

5. Game Tree Representation
Consider the following tree:



A more detailed tree:



---

6. Variables, Domains and Constraints
Not Applicable.
This is a game-tree search problem.

7. Heuristic Function
For this small tree, heuristic evaluation is not required because all terminal utility values are known.
For larger games, a heuristic evaluation function can estimate the utility of non-terminal states.

8. Algorithm Design / Pseudocode

Minimax
MINIMAX(node, maximizingPlayer)

1. If node is terminal:
return utility(node).

2. If node belongs to MAX:
best = -∞
For every child:
best = max(best, MINIMAX(child, MIN))
return best.

3. If node belongs to MIN:
best = +∞
For every child:
best = min(best, MINIMAX(child, MAX))
return best.

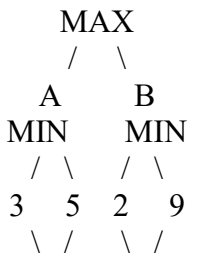
Alpha-Beta Pruning
ALPHA-BETA(node, α, β, maximizingPlayer)

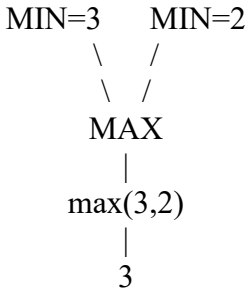
1. If node is terminal:
return utility(node).

2. If MAX:
value = -∞
For each child:
value = max(value, ALPHA-BETA(child, α, β, MIN))
α = max(α, value)
If α >= β:
break
return value.

3. If MIN:
value = +∞
For each child:
value = min(value, ALPHA-BETA(child, α, β, MAX))
β = min(β, value)
If β <= α:
break
return value.

9. Game Tree / Algorithm Diagram
Minimax Evaluation

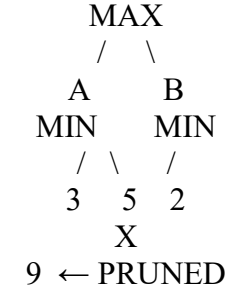




Therefore:
[A=Min(3,5)=3]
[B=Min(2,9)=2]
MAX chooses:
[Max(3,2)=3]
Thus:[\boxed{A\text{ is the best move}}]

---

10. Working / Execution Trace
Minimax
For A:
[Min(3,5)=3]
For B:
[Min(2,9)=2]
At root:
[Max(3,2)=3]
Therefore MAX selects:[\boxed{A}]
Alpha-Beta Pruning
Start with:
[\alpha=-\infty,\quad\beta=+\infty]
Evaluate A:
3 → 5
MIN chooses:
[A=3]
At root:
[\alpha=3]
Now evaluate B.
First child:
[2]
At B:[\beta=2]
Since:[\beta \leq \alpha]
[2\leq3]
the remaining child **9 is pruned**.
Therefore:



Final Result

[\boxed{A}]
Final utility:
[\boxed{3}]
Node Comparison
Without pruning:

- Root = 1
- A = 1
- A leaves = 2
- B = 1
- B leaves = 2

Total:
[1+1+2+1+2=\boxed{7}]
With Alpha-Beta pruning:

- Root = 1
- A = 1
- A leaves = 2
- B = 1
- B first leaf = 1

Total:
[1+1+2+1+1=\boxed{6}]
Therefore:
[\boxed{1\text{ node is pruned}}]
For larger game trees, Alpha-Beta pruning can provide much larger savings.

11. Complexity Analysis
Minimax
Time complexity:
[O(b^d)]
Space complexity:
[O(bd)]
for depth-first implementation.
Alpha-Beta
Worst case:
[O(b^d)]
Best case with excellent move ordering:
[O(b^{d/2})]
Thus Alpha-Beta can effectively allow the AI to search approximately twice as deep in favorable cases.

12. Advantages and Limitations
Minimax Advantages

- Provides optimal decision against an optimal opponent.
- Simple and systematic.
- Suitable for deterministic two-player games.

Minimax Limitations

- Explores many nodes.
- Becomes expensive for deep game trees.

Alpha-Beta Advantages

- Reduces unnecessary node evaluations.
- Produces the same decision as Minimax.

- Allows deeper search.

Alpha-Beta Limitations

- Efficiency depends on move ordering.
- Does not eliminate the fundamental exponential nature of game-tree search.

13. Conclusion

Minimax selects the best move assuming an optimal opponent. Alpha-Beta pruning improves Minimax by removing branches that cannot influence the final decision. In the example, MAX selects move A with utility 3, while the leaf value 9 can be safely pruned.

QUESTION 5 – ONLINE SEARCH AGENT FOR A DYNAMIC WAREHOUSE ROBOT

1. Problem Statement

A warehouse robot must move from its current position to a destination. However, the environment is dynamic and obstacles may appear unexpectedly.
A precomputed path may become invalid when a new obstacle appears.
Therefore, an Online Search Agent is suitable because it can:

1. Observe the environment.
2. Take an action.
3. Detect environmental changes.
4. Update its knowledge.
5. Re-plan the route.

2. Problem Analysis

Initial State: Robot at warehouse starting position S.
Goal State: Delivery location G.
Environment: Warehouse containing shelves, paths and moving obstacles.

Assumptions

- Robot has sensors.
- Robot can detect nearby obstacles.
- The environment may change.
- Robot can calculate a new route.
- The robot maintains information about previously visited states.

Constraints

- Robot cannot move through obstacles.
- Robot should avoid collisions.
- Robot should reach the goal efficiently.
- New obstacles must be handled dynamically.

3. AI Technique Selection

The selected approach is:

[\boxed{\text{Online Search Agent}}]

An online search agent does not know the complete future environment.
Instead, it learns about the environment while acting.

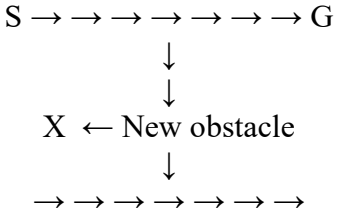
4. Justification of Algorithm

A static informed search such as A* can generate an optimal route when the environment is known and does not change.However, in a dynamic warehouse, a previously generated path can become invalid.
An Online Search Agent is better because it repeatedly:

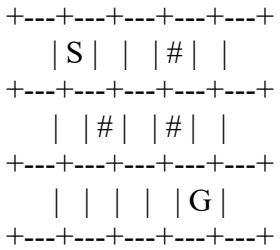
[\text{Sense}\rightarrow\text{Plan}\rightarrow\text{Act}\rightarrow\text{Learn}]
This allows the robot to adapt when an unexpected obstacle appears.

---

5. State Space Representation
Example warehouse:



Another representation:



Here:

- S = Start
- G = Goal
- **= obstacle**
- Empty cells = traversable locations

If an obstacle suddenly appears on the planned route, the robot must re-plan.

6. Variables, Domains and Constraints
Variables
Robot position:
[R=(x,y)]
Domain
Possible robot locations are the free cells in the warehouse.
Constraints

- Robot cannot enter blocked cells.
- Robot must remain inside warehouse boundaries.
- Robot should avoid collisions.
- Robot should eventually reach G.

7. Heuristic Function
A heuristic may be used for local route selection.
For a grid environment, Manhattan distance is commonly used:
[h(n)=|x_n-x_G|+|y_n-y_G|]
It estimates the number of grid moves needed to reach the goal.
However, unlike static A*, the robot must update its knowledge when the environment changes.

8. Algorithm Design / Pseudocode
ONLINE-ROBOT-SEARCH(S, G)

1. CurrentState = S.

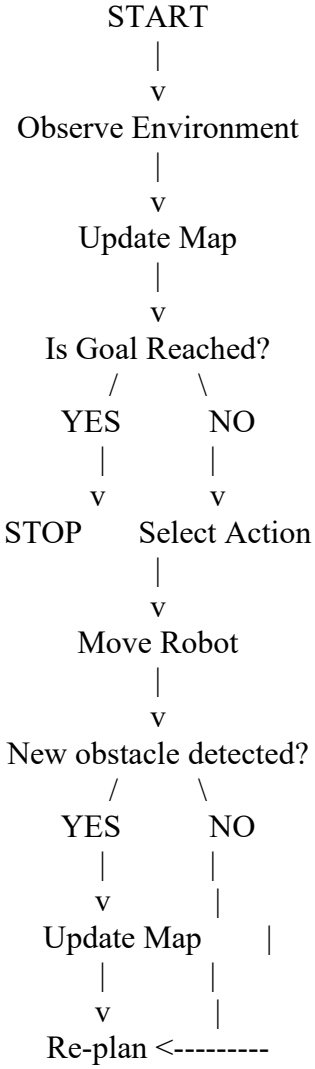
2. Initialize Map with known environment information.
3. While CurrentState != G:
a. Sense the environment.
b. Update Map with newly detected obstacles.
c. Generate possible next states.
d. Remove blocked states.
e. Calculate the best available next move.
f. Move the robot to the selected state.
g. If a new obstacle blocks the planned route:
stop movement
update Map
re-plan the route.

h. Continue until G is reached.

4. Return successful path.

---

9. Algorithm Workflow



---

10. Working / Execution Trace

Suppose the robot initially plans:

[S\rightarrow A\rightarrow B\rightarrow C\rightarrow G]
Step 1
Robot starts at S.
S → A → B → C → G
The route is clear.
Step 2
Robot moves:
[S\rightarrow A]
Step 3
An unexpected obstacle appears at B.
S → A → X → C → G
↑
New obstacle
The robot detects the obstacle using sensors.
Step 4
The robot updates its map:
B = BLOCKED
The original path cannot be used.
Step 5
The robot searches for an alternative route:
[
A\rightarrow D\rightarrow E\rightarrow G
]
Therefore the robot changes its route to:
[
\boxed{S\rightarrow A\rightarrow D\rightarrow E\rightarrow G}
]
Adaptation Cycle

Sense
↓
Detect obstacle
↓
Update environment model
↓
Re-plan
↓
Take alternative action
↓
Continue toward goal

This demonstrates how an Online Search Agent adapts to unexpected changes.

---

11. Complexity Analysis
If the robot performs a search over (N) reachable states:
Time Complexity
A search operation can require approximately:
[O(N)]
depending on the search strategy used for re-planning.
If re-planning occurs (k) times:
[O(kN)]
in the simplified worst-case model.

Space Complexity
The robot may need to store its discovered environment:
[O(N)]

---

12. Comparison with Informed Search

Feature	Online Search Agent	A* / Informed Search
Environment	Partially unknown/dynamic	Usually known
Adaptation	High	Limited unless re-run
Planning	During execution	Usually before execution
Obstacles	Can react to new obstacles	Requires updated map/re-planning
Optimality	Not always guaranteed	A* can guarantee optimality with suitable heuristic
Memory	Depends on explored environment	Can be high
Suitability	Dynamic environments	Static or known environments

Critical Analysis
For a static warehouse, A* may be more efficient because the entire map can be used to calculate an optimal route.
For a dynamic warehouse, an Online Search Agent is more flexible because it can react to changes.
A practical warehouse system could combine online adaptation with heuristic pathfinding such as A* or another incremental replanning method.

---

13. Advantages and Limitations
Advantages
- Handles unknown environments.
- Adapts to newly detected obstacles.
- Does not require complete knowledge beforehand.
- Suitable for robots and autonomous systems.

Limitations
- May not produce a globally optimal route.
- Re-planning can increase computational cost.
- Robot may temporarily take inefficient routes.
- Performance depends on sensor accuracy.

---

14. Conclusion
An Online Search Agent is highly suitable for a warehouse robot operating in a dynamic environment. Instead of relying on a fixed route, the robot continuously observes its surroundings and modifies its plan when obstacles appear. This makes the system robust and adaptive.
